

EcoBot — AI-Powered Waste Classification & Recycling Assistant

Version 2.0 | Author: Dinesh Kumar C | May 2026 | Status: Draft

1. Product Overview

1.1 Vision

EcoBot is a multimodal AI assistant that helps individuals and organizations classify waste correctly, understand proper disposal methods, locate nearby recycling facilities, and understand the environmental impact of their waste — all through text, image, or voice input. It combines a fine-tuned LLaMA 3 model with a RAG pipeline to deliver accurate, context-aware recycling guidance with a strong focus on Indian waste practices.

1.2 Problem Statement

Most people do not know how to segregate waste correctly. A plastic-lined coffee cup goes into recycling bins incorrectly. E-waste gets dumped into general bins. Hazardous items like batteries or syringes are disposed of without caution. Existing solutions are either too generic (static FAQs) or too complex (municipal portals). There is no intelligent assistant that understands context, handles Indian-specific waste items, and makes correct disposal as easy as taking a photo or asking a question.

1.3 Goals

- Classify any waste item from text, image, or voice with high accuracy
- Tell the user exactly which bin to use, why, and how to prepare the item
- Surface environmental impact data to build user awareness
- Show nearby verified recycling and disposal facilities
- Handle India-specific waste items that no public dataset covers
- Demonstrate fine-tuned LLM + RAG + vision + voice — strong portfolio project

1.4 Non-Goals (V1)

- Real-time government or municipal API integration
- Real-time facility availability or appointment booking
- Carbon credit tracking or rewards system (planned for V2)
- Mobile native app

2. User Personas

Persona	Profile	Core Need
Confused Household User	Ravi, 35, working professional	Quickly find out which bin a specific item belongs in
Eco-Conscious Student	Meera, 21, environmental science	Understand environmental impact, learn correct practices

Persona	Profile	Core Need
Facility Manager	Suresh, 45, office building manager	Classify bulk waste, find certified disposal vendors nearby
Parent	Anitha, 38, mother of two	Teach kids waste segregation in a simple, interactive way

3. Input Modes

Mode 1 — Text Input

User types a description of the waste item. Can be a single word or a full sentence.

- "Old smartphone"
- "I have a half-eaten sandwich in a plastic wrapper"
- "Empty motor oil container"

Mode 2 — Image Input (LLaMA 4 Scout via Groq)

User uploads or captures a photo. LLaMA 4 Scout on Groq identifies the waste item directly from the image — no separate vision model or CLIP step needed. One model call handles both identification and classification context.

- Accepts JPG, PNG, WEBP
- Model: meta-llama/llama-4-scout-17b-16e-instruct on Groq API
- Bot confirms identification before classifying: "This looks like a lithium battery — is that correct?"
- Falls back gracefully if image is too blurry: "I cannot identify this clearly — can you describe it in text?"

Why LLaMA 4 Scout over LLaMA 3.2 Vision: Native multimodal, stronger visual reasoning, already on Groq so no extra API key. No CLIP preprocessing step needed.

Mode 3 — Voice Input (Gemini API)

User speaks a description. Gemini API transcribes the audio to text, then the standard text classification pipeline runs.

- API: Google Gemini 2.0 Flash — audio file sent, clean text returned
- Supports English, Hindi, Tamil in V1
- Bot displays transcribed text for user to confirm before processing
- Simple one-call integration — not streaming, just transcription

Why Gemini API over Whisper v3: Gemini handles Indian language accents better and requires no local model setup. Whisper needs to run locally or via a separate API. Gemini is one unified API call.

V2 — Gemini Live (Full Voice-to-Voice)

In V2, Gemini Live replaces the transcription-only approach with a real-time bidirectional audio stream. The user speaks and EcoBot speaks back — no typing at all. This is a significant UX upgrade but adds streaming complexity, so it is planned for after V1 is stable.

4. Core Features (MVP)

F1 — Waste Classification Engine

The core feature. Given any waste item via text, image, or voice, EcoBot classifies it and returns structured output.

Waste categories:

- Wet waste — food scraps, organic material, garden waste
- Dry waste — paper, cardboard, clean plastics, metals, glass
- Hazardous waste — batteries, chemicals, paint, pesticides, medical waste
- E-waste — phones, laptops, cables, PCBs, appliances
- Sanitary waste — diapers, sanitary pads, medical consumables
- Construction & demolition — cement, tiles, wood, debris
- Non-recyclable rejects — multilayer plastics, dirty packaging, thermocol

Output per classification:

- Category name + colour-coded bin label
- One-line reason for the classification
- Confidence level — triggers clarification question if low
- Structured JSON output every time — enforced by fine-tuning

F2 — Step-by-Step Disposal Instructions

After classification, bot provides exact disposal steps tailored to the specific item.

- How to prepare the item before disposal (rinse, disassemble, deflate)
- Which bin or collection point to use
- Whether a special facility is required
- Safety precautions for hazardous items

F3 — Environmental Impact Information

After every classification, bot surfaces one relevant environmental fact to build awareness. Facts are rotated within a session to avoid repetition. Sourced from the curated ChromaDB impact facts collection.

- "A single lithium battery can contaminate 600,000 litres of water if landfilled"
- "Recycling one aluminium can saves enough energy to run a TV for 3 hours"

F4 — Nearby Facility Finder

- User provides city or pincode
- Bot filters SQLite facility database by waste category accepted
- Returns top 3 facilities: name, address, accepted waste types, hours
- V1: Static curated facility dataset in SQLite
- V2: Google Places API for live facility data

F5 — Multi-Turn Conversation with Memory

- "What about the charger that came with it?" — bot understands prior context
- "Where can I drop this off?" — bot knows the item already
- Powered by LangChain ConversationBufferMemory

F6 — Batch Classification

- "Classify: old newspaper, broken earphones, leftover curry, empty paint can"
- Returns a structured table with category and bin for each item

- Highlights hazardous items prominently

5. Advanced Features (V2)

F7 — Eco Score Tracker

- Score increases per correctly classified item in the session
- Gamified: "You've correctly classified 8 items today!"

F8 — Item Lifecycle Explainer

- "Your newspaper will be pulped and turned into new paper within 7 days"
- Closes the loop — builds deeper engagement beyond bin allocation

F9 — Brand / Product Lookup

- "What do I do with an old iPhone 12?" → e-waste with Apple-specific steps
- "I have a Tetra Pak juice box" → non-recyclable, multilayer

F10 — Gemini Live Voice-to-Voice Mode

- Full real-time audio in / audio out — no typing needed
- User speaks, EcoBot classifies and speaks the result back
- Requires streaming WebSocket integration on backend

6. RAG Design — Collections and Content

Overview

EcoBot uses ChromaDB with three separate collections. Each collection has a distinct purpose and is populated differently. The key design principle: only content that benefits from semantic similarity search goes into ChromaDB. Exact lookups (bin colors, facility addresses) stay in SQLite.

Collection 1 — Disposal Guidelines

Detailed disposal instructions per waste category and item type. Retrieved after classification to generate the step-by-step guidance in F2.

- What is stored: preparation steps, bin allocation, safety warnings, special handling notes per item
- Chunk size: 400-500 tokens per item entry
- Metadata tags: category, sub_category, hazardous (bool), special_facility (bool)
- Why RAG and not SQL: disposal instructions are free-form prose, not structured fields. Semantic search retrieves the most relevant guidance even for vague or combined items.

Collection 2 — Environmental Impact Facts

Curated library of environmental impact statistics indexed by waste category and item type. Rotated per session to avoid repetition.

- What is stored: one fact per document, tagged with category and severity
- Example: "Plastic bags take 400-1000 years to decompose. Over 1 trillion are used globally per year."

- ~200 manually written facts — quality controlled, no hallucinations

Collection 3 — Product Knowledge Base

Maps known product brands and packaging types to waste categories and specific disposal notes. Enables fuzzy product name matching for the F9 feature.

- What is stored: product name, brand, category, disposal notes, special instructions
- Example: Tetra Pak juice carton → non-recyclable (multilayer), special drop-off points
- ~200 products manually curated to start, expandable

Runtime Fallback — Exa.ai

When ChromaDB returns a low-confidence result (similarity score below threshold), EcoBot falls back to a live Exa.ai search query to find current disposal guidance for that item. This handles rare or newly-introduced products not yet in the knowledge base.

Exa.ai is a pay-per-query search API that returns clean webpage content. Used only as fallback, not primary retrieval, to keep costs low.

What Does NOT Go Into RAG

- Waste category to bin color mapping — simple SQLite lookup table
- Facility data (name, address, hours) — SQLite, filtered by city and category
- User session history — LangChain ConversationBufferMemory

7. RAG Content Creation Pipeline

There is no public dataset that provides item-level disposal instructions in a structured format. The knowledge base is built by combining web scraping, automated conversion, and manual curation.

Step 1 — Scrape earth911.com (Crawl4AI)

earth911.com is the largest English-language recycling database with item-level disposal pages. Crawl4AI is used — it is open source, free, runs locally, and outputs clean markdown ready for embedding. No API key or rate limit concerns.

```
from crawl4ai import AsyncWebCrawler

async with AsyncWebCrawler() as crawler:
    result = await crawler.arun(
        url='https://earth911.com/recycling-guide/how-to-recycle-lithium-ion-batteries'
    )
    print(result.markdown) # clean text, ready to embed
```

- Target: ~300 item pages covering common waste categories
- Output: clean markdown per item, saved to /data/raw/earth911/

Step 2 — Parse CPCB and Government PDFs (Firecrawl)

Indian government waste guidelines are published as PDFs on cpcb.nic.in and municipal websites. Firecrawl handles PDF parsing and JavaScript-rendered pages that Crawl4AI cannot.

- Source: CPCB Solid Waste Management Rules, municipal corporation guidelines
- Target: ~50 documents covering Indian-specific regulations
- Firecrawl free tier is sufficient for this volume

Step 3 — Auto-Convert to Structured Format (LLaMA 3 70B via Groq)

Scraped markdown is fed to LLaMA 3 70B on Groq to convert into structured JSON disposal entries. This is model distillation — a large model generates clean training data for a smaller model.

```
def convert_to_structured(scraped_text, item_name):
    prompt = f'''
    Convert this recycling page into structured JSON.
    Return only valid JSON, no extra text.

    Source: {scraped_text}

    Output:
    {{
      "item": "{item_name}",
      "category":
"dry_waste|wet_waste|hazardous|e_waste|sanitary|construction|non_recyclable",
      "bin_color": "blue|green|red|grey",
      "recyclable": true/false,
      "reason": "one sentence",
      "preparation_steps": ["step 1", "step 2"],
      "safety_notes": "string or null",
      "special_facility": true/false
    }}
    '''
    # call Groq llama3-70b-8192
```

Step 4 — Manual India-Specific Entries

Earth911 is US-centric. Indian-specific waste items — agarbatti ash, steel tiffins, coconut shells, clay diyas, newspaper raddi, old sarees — are not covered. These are written manually as structured JSON. This is the most valuable part of the knowledge base.

- Target: ~150 India-specific items
- Format: same JSON schema as auto-converted entries
- Examples: agarbatti ash, clay diyas, steel tiffins, coconut shells, puja flowers, old clothes (raddi)

Step 5 — Augment with Input Variations

Each item entry gets 3-4 alternative phrasings to handle how real users actually type. Generated automatically via a Groq call.

- "old smartphone" / "broken phone" / "dead iPhone" / "cracked mobile" — all map to same output
- Augmentation multiplies dataset from ~450 to ~2000 examples

Step 6 — Validate Before Embedding

100 randomly sampled entries are manually reviewed before embedding. Auto-generation produces roughly 10-15% errors — wrong categories, missing steps. These must be fixed before ingestion.

Final Knowledge Base Composition

Source	Method	Count
earth911.com disposal pages	Crawl4AI scrape + Groq auto-convert	~300 entries

Source	Method	Count
CPCB / municipal PDFs	Firecrawl parse + Groq auto-convert	~100 entries
India-specific items	Manually written JSON	~150 entries
Environmental impact facts	Manually written, quality controlled	~200 facts
Product knowledge base	Manually curated JSON	~200 products
Input variations (augmentation)	Auto-generated via Groq	~1500 extras
Total		~2450 entries

8. Fine-Tuning Plan

Why Fine-Tune at All

LLaMA 3 8B with a good system prompt classifies common items reasonably well. Fine-tuning is justified for three specific reasons in EcoBot's case:

- Consistent structured JSON output every single time — base models drift from format under prompt pressure
- India-specific accuracy — agarbatti ash, clay diyas, steel tiffins are not in base model training data
- Smaller faster offline model — fine-tuned 8B via Ollama avoids Groq API dependency for classification

Fine-tuning is for the classification step only. LLaMA 3 70B via Groq still handles final response generation, conversation, and disposal instruction formatting.

Training Dataset Composition

Source	Method	Count
Earth911 scraped pages	Auto-converted via Groq distillation	~300
CPCB / EPA guidelines	Auto-converted via Groq distillation	~100
India-specific items	Manually written instruction pairs	~150
Edge cases (composites, multilayer)	Manually written	~50
Input variations (augmentation)	Auto-generated alternative phrasings	~1500
Total		~2100

Training Data Format

Instruction-following pairs in Alpaca format. The model learns one specific structured task with a fixed output schema.

```
{
  "instruction": "Classify this waste item and provide disposal guidance.",
  "input": "Empty aerosol deodorant can",
  "output": {
```

```

"category": "hazardous",
"bin_color": "red",
"bin_label": "Hazardous Waste",
"recyclable": false,
"reason": "Aerosol cans contain residual pressurised gas – fire risk",
"preparation_steps": [
  "Release all remaining gas before disposal",
  "Do not crush or puncture",
  "Place in hazardous waste bin or collection point"
],
"safety_notes": "Keep away from heat sources before disposal",
"special_facility": false
}
}

```

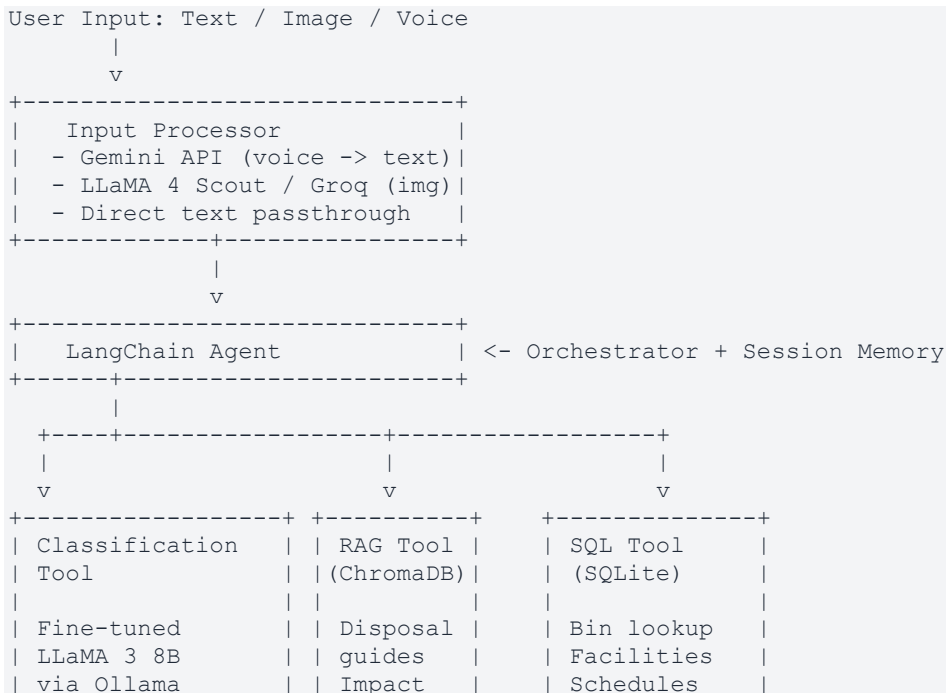
Fine-Tuning Approach

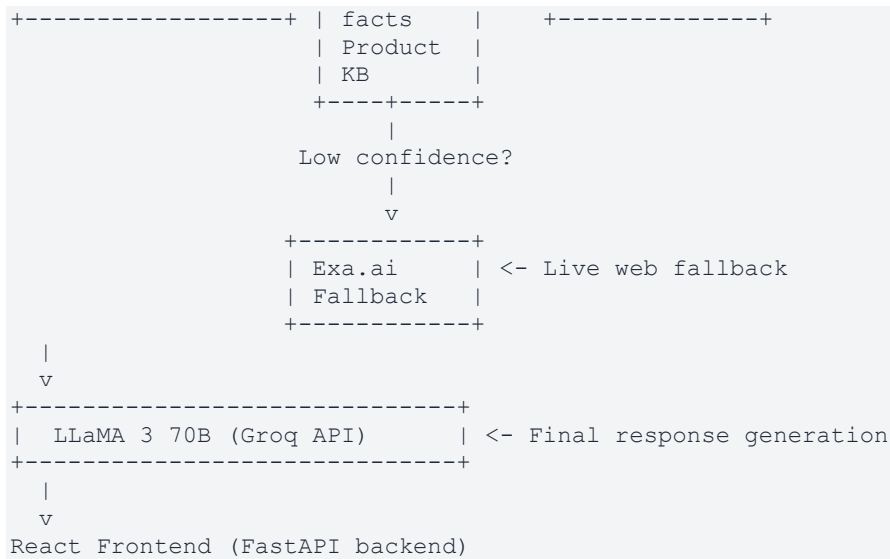
- Method: QLoRA (Quantized LoRA) — runs on free Google Colab T4 GPU
- Base model: LLaMA 3 8B Instruct (meta-llama/Meta-Llama-3-8B-Instruct)
- Framework: HuggingFace PEFT + TRL SFTTrainer
- LoRA rank: 16, alpha: 32, dropout: 0.05
- Training time: ~3-4 hours on Colab T4 for 2100 examples
- Output: LoRA adapter saved, merged with base model, quantized to 4-bit GGUF for Ollama

Evaluation

- Classification accuracy on 200-example held-out test set: target > 90%
- Confusion matrix across all 7 waste categories
- Manual review of 50 Indian-specific edge case items
- JSON format compliance rate: target 100% (fine-tuning enforces this)

9. Technical Architecture





10. Tech Stack

Layer	Technology	Reason
Classification LLM	Fine-tuned LLaMA 3 8B via Ollama	Custom-trained, runs offline, consistent JSON output
Response LLM	LLaMA 3 70B via Groq API	High-quality response generation
Vision (image input)	LLaMA 4 Scout via Groq API	Native multimodal, no separate CLIP step, already on Groq
Voice (transcription)	Google Gemini 2.0 Flash API	Better Indian accent support, simple one-call integration
Voice V2 (real-time)	Google Gemini Live	Full voice-to-voice streaming — planned for V2
Web scraping	Crawl4AI (earth911)	Free, open source, AI-optimized markdown output
PDF / complex page scraping	Firecrawl (CPCB PDFs)	Handles PDFs and JS-rendered pages cleanly
Live fallback search	Exa.ai	Real-time web retrieval for unknown items
Orchestration	LangChain	Agent, memory, tool routing
Vector DB	ChromaDB	Disposal guides, impact facts, product KB
Embeddings	all-MiniLM-L6-v2	Lightweight, accurate semantic search
Structured DB	SQLite	Bin lookup, facility data, collection schedules
Backend	FastAPI	REST API + WebSocket for streaming
Frontend	React + TailwindCSS	Production-quality, portfolio-worthy UI
Fine-tuning	HuggingFace PEFT + TRL (QLoRA)	Efficient fine-tuning on free Colab T4 GPU
Deployment	Docker + HuggingFace Spaces	Free hosting, shareable demo link

11. Data Model

Waste Item (SQLite — exact lookups)

```
id, name, aliases (JSON),
category, sub_category,
bin_color, bin_label,
hazardous (bool), recyclable (bool),
preparation_steps (JSON),
safety_notes, special_facility_required (bool)
```

Facility (SQLite — filtered by city + category)

```
id, name, address, city, pincode,
latitude, longitude,
accepted_categories (JSON array),
operating_hours, contact, verified (bool)
```

Disposal Guideline (ChromaDB Collection 1)

```
id, item_name, category, sub_category,
content (full disposal guide text),
source_url, hazardous (bool), special_facility (bool)
```

Environmental Fact (ChromaDB Collection 2)

```
id, category, item_type,
fact_text, source, severity (low/medium/high)
```

Product KB (ChromaDB Collection 3)

```
id, product_name, brand, model,
category, disposal_notes, special_instructions
```

12. Sample Conversation Flows

Flow 1 — Image Input (LLaMA 4 Scout)

```
User: [uploads photo of a cracked smartphone]
Bot: I can see a smartphone — is that correct?
User: Yes, the whole phone, it is broken
Bot: E-waste | Red bin (dedicated e-waste collection)
Steps:
1. Back up and wipe your data first
2. Remove SIM card and keep it
3. Place in padded bag to prevent battery damage
4. Drop at authorised e-waste centre
Impact: 1 phone recycled recovers enough gold to make a small ring.
Nearest centre: GreenDrop E-waste, Anna Nagar — 2.3 km
```

Flow 2 — Voice Input (Gemini API)

```
User: [speaks] I have an old mosquito repellent coil, half used
Bot: I heard: 'old mosquito repellent coil, half used' — correct?
```

User: Yes
Bot: Hazardous Waste | Red bin
Steps:
1. Keep in original packaging if available
2. Do not mix with food or organic waste
3. Seal in a plastic bag before disposal
Impact: Repellent coils contain pyrethrin compounds,
harmful to aquatic life if they reach waterways.

Flow 3 — India-Specific Item

User: What do I do with clay diyas after Diwali?
Bot: Clay diyas are Wet Waste | Green bin
Steps:
1. Allow to cool and clean off wax residue
2. Break into smaller pieces if needed
3. Can be composted – clay enriches soil
4. Alternatively, crush and mix into garden soil directly
Impact: Clay is fully biodegradable. Unlike plastic decor,
clay diyas return to soil within weeks.

13. React Frontend — Key Screens

Screen 1 — Input Screen

- Three large input buttons: Type, Upload Photo, Speak
- Recent classifications shown as chips below input area
- Eco score badge in the top-right corner

Screen 2 — Classification Result Card

- Large colour-coded bin label — green / blue / red / grey
- Item name + confidence badge (High / Medium / Low)
- Expandable sections: Disposal Steps, Environmental Impact, Nearby Facilities
- Image preview shown if input was a photo
- Voice transcription shown if input was audio

Screen 3 — Chat Interface

- Standard chat bubble UI for follow-up questions
- Voice input button in message bar (Gemini API)
- Image upload as attachment icon (LLaMA 4 Scout)
- Transcription preview before sending voice input

14. Success Metrics

Metric	Target
Classification accuracy (fine-tuned model)	> 90% on test set
JSON format compliance	100% — enforced by fine-tuning
Vision identification accuracy (LLaMA 4 Scout)	> 85%

Metric	Target
Voice transcription accuracy (Gemini API)	> 95% English, > 88% Hindi/Tamil
Response latency — text input	< 2 seconds
Response latency — image input	< 5 seconds
RAG retrieval relevance (ChromaDB)	> 85%
Exa.ai fallback trigger rate	< 10% of queries
Hallucination rate	< 3%

15. Folder Structure

```

ecobot/
├── data/
│   ├── raw/
│   │   ├── earth911/          <- Crawl4AI scraped markdown
│   │   └── cpcb_pdfs/         <- Firecrawl parsed PDFs
│   ├── processed/
│   │   ├── disposal_guides.json
│   │   ├── india_specific.json
│   │   ├── env_facts.json
│   │   └── product_kb.json
│   ├── finetuning/
│   │   ├── train.jsonl        <- 1800 instruction pairs
│   │   └── test.jsonl         <- 200 held-out examples
│   ├── facilities.csv
│   └── waste_items.csv
├── embeddings/
│   └── chroma_db/
│       ├── disposal_guides/
│       ├── env_facts/
│       └── product_kb/
├── scraping/
│   ├── crawl_earth911.py      <- Crawl4AI scraper
│   ├── crawl_cpcb.py          <- Firecrawl PDF parser
│   ├── convert_to_json.py     <- Groq distillation script
│   └── augment_dataset.py     <- Input variation generator
├── fine_tuning/
│   ├── train.py               <- QLoRA training script
│   └── evaluate.py
├── backend/
│   ├── main.py                <- FastAPI app
│   ├── agent.py               <- LangChain agent
│   └── tools/
│       ├── classify_tool.py    <- Fine-tuned model via Ollama
│       ├── rag_tool.py        <- ChromaDB retrieval
│       ├── sql_tool.py        <- Bin + facility lookup
│       ├── vision_tool.py     <- LLaMA 4 Scout on Groq
│       ├── voice_tool.py      <- Gemini API transcription
│       └── exa_fallback.py     <- Exa.ai live search
│   ├── memory.py
│   └── prompts.py
├── frontend/
│   └── src/
│       ├── components/
│       │   ├── InputPanel.jsx
│       │   ├── ResultCard.jsx
│       │   ├── ChatInterface.jsx
│       └── FacilityList.jsx

```

```

├── App.jsx
├── package.json
├── tests/
│   └── test_classification.py
├── requirements.txt
├── Dockerfile
└── README.md

```

16. Development Milestones

Phase	Deliverable	Timeline
Phase 1	Crawl4AI scrapes earth911 + Firecrawl parses CPCB PDFs	Day 1-2
Phase 2	Groq distillation + manual India entries → structured JSON	Day 3-4
Phase 3	Augmentation + validation → embed into ChromaDB	Day 5
Phase 4	QLoRA fine-tune on Colab + evaluate on test set	Day 6-8
Phase 5	FastAPI backend + LangChain agent + text classification	Day 9-10
Phase 6	LLaMA 4 Scout vision tool + Gemini API voice tool	Day 11-12
Phase 7	React frontend — all 3 screens	Day 13-14
Phase 8	Exa.ai fallback + facility finder + batch mode	Day 15
Phase 9	Docker + HuggingFace Spaces deployment	Day 16

17. Out of Scope (V1)

- Real-time municipal API or Google Maps integration
- User authentication and persistent eco score history
- Carbon credit calculation or rewards system
- Gemini Live voice-to-voice (planned V2)
- IoT sensor integration for smart bins
- Mobile native app